

Multi-agent Debugging — Worked Example

[Back to the exercise](#) · [Round 1: Basic](#) · [Round 2: Intermediate](#) · [Round 3: Real-world](#) · [Round 4: Multi-agent](#) · [Solution](#)

One scenario, followed across all 4 rounds: Doc11's Project 4 supervisor (`agents/supervisor.py`), routing to Research → Analysis → Writer → Reviewer. For this category, "Round 4: Multi-agent" means the failure is only visible running the *entire* 4-agent pipeline together — not testing the supervisor plus one specialist in isolation. Work through the rounds in order. Each ends with "**What do you think is wrong?**" — stop and actually answer before reading on.

- [Round 1: Basic](#)
- [Round 2: Intermediate](#)
- [Round 3: Real-world](#)
- [Round 4: Multi-agent](#)

[Back to the exercise](#) · [Round 1: Basic](#) · [Round 2: Intermediate](#) · [Round 3: Real-world](#) · [Round 4: Multi-agent](#) · [Solution](#)

Round: Basic

Setup: the supervisor node hands off to a specialist using `Command` :

```
def supervisor_node(state):
    next_agent = decide_next_agent(state)
    return Command(goto=next_agent)
```

`decide_next_agent()` returns one of a fixed set of strings, one of which has a typo baked in:

```
return "Reserach"
```

Symptoms: fails immediately, every single run that routes to the research specialist — not intermittent.

Error output:

```
Traceback (most recent call last):
  File "project_4_multi_agent_system/agents/supervisor.py", line 22, in supervisor_node
    return Command(goto=next_agent)
  ...
langgraph.errors.InvalidUpdateError: Node 'Reserach' referenced in Command(goto=next_agent) does not exist in the graph. Available nodes: ['supervisor', 'research_agent', 'analysis_agent', 'writer_agent', 'reviewer_agent']
```

Expected vs. actual: - Expected: `Command(goto="research_agent")` hands control to the Research agent. - Actual: the graph fails immediately, because no node named `"Reserach"` exists.

What do you think is wrong?

[Back to the exercise](#) · [Round 1: Basic](#) · [Round 2: Intermediate](#) · [Round 3: Real-world](#) · [Round 4: Multi-agent](#) · [Solution](#)

Round: Intermediate

Setup: routing is fixed — the Research agent runs correctly and writes its output:

```
def research_node(state):
    finding = run_research_tools(state["task"])
    return Command(goto="analysis_agent", update={"finding": finding})
```

The shared state schema, defined separately, actually declares a different field name:

```
class SharedState(TypedDict):
    task: str
    research_findings: str
    analysis: str
    draft: str
```

Symptoms: no exception anywhere. The graph runs start to finish and produces a final answer — just a noticeably weak, generic one, every time.

Log output:

```
[research_agent] finding = "The refund policy allows returns within 30 days of pu
[analysis_agent] state.research_findings = ""
[analysis_agent] WARNING: no research findings to analyze, proceeding with genera
[writer_agent] draft: "Our return policy details may vary; please check with supp
```

Expected vs. actual: - Expected: the Analysis agent reads what Research actually found and builds on it. - Actual: the Analysis agent's view of `research_findings` is always an empty string, even though Research clearly found something real, one line above in the same log.

What do you think is wrong?

[Back to the exercise](#) · [Round 1: Basic](#) · [Round 2: Intermediate](#) · [Round 3: Real-world](#) · [Round 4: Multi-agent](#) · [Solution](#)

Round: Real-world

Setup: the field-name bug is fixed. The supervisor picks between two specialists using an LLM call, given each specialist's description:

```
SPECIALISTS = {
    "research_agent": "Finds factual information and data to answer a question.",
    "analysis_agent": "Analyzes information and data to draw conclusions.",
}
```

Symptoms: only sometimes happens — most requests route to the obviously-correct specialist. A specific style of request routes inconsistently: the same *kind* of task, worded slightly differently, sometimes goes to Research and sometimes to Analysis.

Log output (5 similar requests, same underlying task):

```
[supervisor] task: "find and interpret last quarter's revenue numbers"    -> rout
[supervisor] task: "look up and interpret last quarter's revenue numbers" -> rout
[supervisor] task: "get and interpret last quarter's revenue figures"      -> rout
[supervisor] task: "find last quarter's revenue numbers and interpret them" -> ro
[supervisor] task: "interpret last quarter's revenue numbers after finding them"
```

Expected vs. actual: - Expected: a supervisor routing decision is a deliberate design choice with a knowable reason, not a coin flip. - Actual: for tasks that genuinely need both "finding" and "interpreting," the supervisor's routing flips unpredictably between the two specialists depending on exact wording — because both specialist descriptions genuinely apply, and the supervisor has no tiebreaker.

What do you think is wrong?

[Back to the exercise](#) · [Round 1: Basic](#) · [Round 2: Intermediate](#) · [Round 3: Real-world](#) · [Round 4: Multi-agent](#) · [Solution](#)

Round: Multi-agent

This round only shows up running the full 4-agent pipeline — not the supervisor alone, not any 2-agent pair alone.

Setup: the Round 2 field-name bug got "fixed" quickly under deadline pressure by adding a second field instead of renaming the first one:

```
class SharedState(TypedDict):
    task: str
    research_findings: str    # never actually written to, still
    finding: str              # what research_node actually writes to
    analysis: str
    draft: str
    review_passed: bool
    revision_count: int
```

`analysis_node` was updated to read from `finding` and works correctly now. `reviewer_node`, written independently by someone checking the draft against the *original* field name, wasn't:

```
def reviewer_node(state):
    if state["research_findings"] not in state["draft"]:
        return Command(goto="writer_agent", update={
            "review_passed": False,
            "revision_count": state["revision_count"] + 1,
        })
    return Command(goto=END, update={"review_passed": True})
```

Symptoms: testing Research + Analysis together: works perfectly, `finding` flows through correctly. Testing Writer + Reviewer together with a hand-written fake `draft` and a hand-written fake `research_findings` that match: works perfectly. Running all 4 agents together on a real task: the Reviewer rejects every draft, forever, until it hits the revision cap and gives up.

Log output:

```
[research_agent] finding = "Q1 revenue grew 12% year over year."
[analysis_agent] analysis = "Growth is driven mainly by the new product line."
[writer_agent] draft: "Q1 revenue grew 12% year over year, driven mainly by the n
```

```
[reviewer] state.research_findings = ""    (still the unused field)
[reviewer] REJECTED: draft does not reflect research findings
[writer_agent] revision #2: draft: "Q1 revenue grew 12% year over year, driven ma
[reviewer] REJECTED: draft does not reflect research findings
... (continues to revision_count = 3, then gives up) ...
```

Expected vs. actual: - Expected: the Reviewer checks the draft against the same findings the Writer actually used, and approves a genuinely accurate draft. - Actual: the Writer and Analysis agents were fixed to use `finding`, but the Reviewer still reads the old, never-written `research_findings` field — which is always empty — so its check `( state["research_findings"] not in state["draft"] )` is comparing the draft against an empty string, which technically is "in" any string, except the actual check is written the other way and always fails. Neither the Research+Analysis pair test nor the Writer+Reviewer pair test (with matching hand-written fixtures) could ever catch this, because each pair test quietly used whichever field name its own author happened to pick.

What do you think is wrong?

[Back to the exercise](#) · [Round 1: Basic](#) · [Round 2: Intermediate](#) · [Round 3: Real-world](#) · [Round 4: Multi-agent](#) · [Solution](#)

Full solution: [Show me the solution](#)